

ESP32 <-> Raspberry Pi UART Setup Guide

Files

- `esp_mecanum_uart.ino`: main ESP32 mecanum controller over UART
- `pi_mecanum_commander.py`: Pi-side command sender and telemetry reader
- `esp_uart_ping_test.ino`: minimal ESP32 UART test sketch
- `pi_uart_ping_test.py`: minimal Pi UART test script

Important Electrical Note

You said you are using ESP32 `TX0/RX0` on a 38-pin ESP32-WROOM dev board.

That is electrically valid with the Raspberry Pi UART because both sides are 3.3V TTL.

The required wiring is:

1. ESP32 `TX0` -> Raspberry Pi `RX`
2. ESP32 `RX0` -> Raspberry Pi `TX`
3. ESP32 `GND` -> Raspberry Pi `GND`

Do **not** connect 5V between UART pins.

Practical Warning About UART0

Using `TX0/RX0` means you are sharing the same UART used for USB programming and boot logs.

That means:

1. The ESP32 will emit boot noise on power-up.
2. The Pi script must ignore non-JSON junk at startup. The provided scripts already do that.
3. Uploading can fail if the Pi is actively driving the ESP32 RX line.

If upload fails, temporarily disconnect the Pi wire from ESP32 `RX0` while flashing.

Raspberry Pi Ubuntu Configuration

Run these steps on the Pi.

1. Check what UART device you have

```
ls -l /dev/serial0
ls -l /dev/ttyAMA0
ls -l /dev/ttyS0
```

Use `/dev/serial0` in the Python files unless you know your Pi uses a different device.

2. Disable the serial login console

Check:

```
sudo systemctl status serial-getty@ttyAMA0.service
sudo systemctl status serial-getty@ttyS0.service
```

Disable whichever one exists and is active:

```
sudo systemctl stop serial-getty@ttyAMA0.service
sudo systemctl disable serial-getty@ttyAMA0.service
```

or

```
sudo systemctl stop serial-getty@ttyS0.service
sudo systemctl disable serial-getty@ttyS0.service
```

3. Enable UART in boot config

Open:

```
sudo nano /boot/firmware/config.txt
```

Make sure this line exists:

```
enable_uart=1
```

If Bluetooth is stealing the main UART on your Pi model, also add:

```
dtoverlay=disable-bt
```

Then disable Bluetooth service if needed:

```
sudo systemctl disable hciuart
```

4. Remove serial console from kernel command line

Open:

```
sudo nano /boot/firmware/cmdline.txt
```

If you see something like this, remove it:

```
console=serial0,115200
```

Keep everything else on the same line.

5. Reboot

```
sudo reboot
```

6. Install Python serial package

```
sudo apt update
sudo apt install -y python3-pip
python3 -m pip install pyserial
```

7. Give your user serial permission if needed

```
sudo usermod -a -G dialout $USER
```

Then log out and back in, or reboot again.

First Bring-Up Procedure

Do this in order.

A. Test the raw UART link first

1. Flash `esp_uart_ping_test.ino` to the ESP32.
2. Copy `pi_uart_ping_test.py` to the Pi.
3. Edit the `PORT` variable if your UART device is not `/dev/serial0`.
4. Run:

```
python3 pi_uart_ping_test.py
```

Expected result:

1. You see a **ready** JSON line.
2. You see a **pong** JSON line.
3. You see a **hello** JSON line.
4. The script prints **UART ping test passed**.

If this fails, do **not** move on yet.

B. Flash the real controller

1. Flash **esp_mecanum_uart.ino**.
2. Keep the same UART wiring.
3. Make sure the Pi script still points to the correct serial device.

C. Run the commander script

1. Open **pi_mecanum_commander.py**.
2. Set **MODE = "sequence"** to run the list automatically, or **MODE = "interactive"** to type commands.
3. Run:

```
python3 pi_mecanum_commander.py
```

Command Reference

These are the commands the ESP32 accepts.

MOVE

Format:

```
MOVE seq=1 angle=0 dist=60 speed=28 heading=0 timeout=15000
```

Meaning:

1. **angle**: body-frame move direction in degrees
2. **dist**: distance in cm
3. **speed**: translational cruise RPM cap
4. **heading**: heading to hold during the move
5. **timeout**: optional turn-phase timeout in ms

Direction convention:

1. **0** = forward
2. **90** = left strafe
3. **-90** = right strafe
4. **180** = backward
5. **45** = forward-left
6. **-45** = forward-right

Behavior:

1. The ESP first turns to the requested heading.
2. Then it drives in the requested mecanum direction.
3. It keeps the heading with the IMU while translating.

TURN

Format:

```
TURN seq=2 heading=90 speed=12 timeout=12000
```

This performs an in-place heading move only.

TWIST

Format:

```
TWIST seq=3 forward=10 strafe=0 turn=0 timeout=1000
```

This is a timed direct body command in RPM space.

Use it for quick manual tests or teleop-like motion.

STOP

Format:

```
STOP seq=4
```

Stops the robot immediately.

ZERO_IMU

Format:

```
ZERO_IMU seq=5
```

Zeros the yaw estimate.

RESET_ENC

Format:

```
RESET_ENC seq=6
```

Resets encoder-based position tracking.

STATUS

Format:

```
STATUS seq=7
```

Returns one telemetry packet immediately.

Telemetry Packet Meaning

The ESP sends JSON lines periodically.

Important fields:

1. `mode`: `idle`, `turning`, `driving`, or `twist`
2. `pose.forwardCm`: encoder-derived forward displacement
3. `pose.strafeCm`: encoder-derived strafe displacement
4. `pose.progressCm`: progress along the current move command direction
5. `pose.remainingCm`: remaining distance along the commanded move direction
6. `pose.crossTrackCm`: drift sideways relative to the current commanded move direction
7. `imu.yawDeg`: current yaw estimate
8. `imu.gyroDps`: current yaw rate
9. `rpm`: measured wheel RPMs in motor order `[BR, FR, BL, FL]`
10. `targetRpm`: commanded wheel RPMs in the same order
11. `pwm`: applied PWM values in the same order

What Was Preserved From Your Final Tuning Sketch

This controller keeps the important finalized control values from your original ESP code:

1. Wheel PID gains: `Kp=50`, `Ki=6`, `Kd=0`
2. Position PID gains: `Kp=0.05`, `Ki=0.0018`, `Kd=0`
3. Heading move PID gains: `Kp=1.95`, `Ki=0.01`, `Kd=0.08`
4. Heading hold PID gains: `Kp=0.65`, `Ki=0.08`, `Kd=1.20`
5. `COUNTS_PER_WHEEL_REV = 4346.8`
6. `wheelDiameterCm = 9.7`
7. The same breakaway PWM behavior and yaw-hold wheel scheduling
8. The same turn-first then move behavior for discrete moves

Troubleshooting

No response from ESP

Check:

1. RX/TX are crossed correctly.
2. Grounds are common.
3. UART device path on the Pi is correct.
4. Serial console is disabled on Ubuntu.
5. Baud rate is **115200** on both sides.

Garbage characters or random startup junk

That is expected on UART0 because ESP32 boot ROM prints there.

The provided Python scripts already ignore non-JSON lines.

Flashing stops working

Temporarily disconnect the Pi wire from ESP32 **RX0** while uploading.

Strafe direction is mirrored

Mecanum wheel roller orientation sometimes differs between builds.

If forward works but left/right is reversed, change the sign convention in **computeMoveMix()** inside **esp_mecanum_uart.ino** and re-test.

Recommended First Real Test

After ping test passes, use these in **pi_mecanum_commander.py**:

1. Forward 30 cm at heading 0
2. Left strafe 20 cm at heading 0
3. Turn to 90 degrees
4. Forward-right diagonal 20 cm while holding 90 degrees

That sequence tells you quickly whether forward, strafe, diagonal, and turn all behave correctly.